

SUPPLEMENTARY · MODULE 8

Testing Spring Boot Microservices

Test at the right layer: unit, slice, contract, and Testcontainers so CI proves the same paths
OpenShift will run.



THE MICROSERVICES TESTING STRATEGY · TESTING PYRAMID VS. TESTING TROPHY · UNIT TESTING WITH JUNIT 5



WHY A STRATEGY MATTERS

- Microservices increase the number of components, interactions, and failure points
- A layered testing strategy reduces risk and optimizes feedback speed
- Fast feedback at lower levels, confidence at higher levels



TESTING PRINCIPLES

- Focus on behavior, not implementation details; test in isolation with clear scope
- Shift left to find defects early; automate for repeatable, reliable results



TESTING PYRAMID

- Classic model with a wide base of unit tests (70-80% unit, 15-20% integration, 5-10% contract/E2E)
- Best for stable teams, well-defined contracts, and systems needing fast CI/CD feedback



TESTING TROPHY

- More emphasis on higher-level tests (20-30% unit, 20-30% integration, 20-30% contract, 30-40% E2E)
- Best for new/rapidly changing products where user value matters more than internals



KEY JUNIT 5 FEATURES

- Modular architecture built on the Jupiter API, extensible
- **Simple annotations:** @Test, @DisplayName, @BeforeEach, @AfterEach, @Nested
- Powerful assertions, assumptions for conditional tests, dynamic and parameterized tests



BEST PRACTICES

- Test one behavior at a time; keep tests small, focused, and fast
- Use meaningful test and method names; avoid test dependencies and side effects

KEY TAKEAWAY Unit, slice, integration, contract, and end-to-end tests each answer a different question at a different cost. • Stable, contract-driven systems suit the pyramid's unit-heavy mix; fast-changing products suit the trophy's integration-heavy mix. • @Test, @DisplayName, assertions, and parameterized tests turn one behavior into one clear, fast, repeatable test case.

Unit Testing with JUnit / Mockito



JUnit (Testing Framework)



Write tests

Create test methods for each unit of code



Run tests

Execute tests and get results



Assertions

Verify expected outcomes



Reports

View test results and coverage



Mockito (Mocking Framework)



Create Mocks

Mock dependencies and external collaborators



Define Behavior

Stub method calls and define return values / exceptions



Verify Interactions

Verify method calls and interactions



Isolate Unit

Test unit in isolation from external dependencies



Example

Service: UserService

```
public class UserService {  
    private final UserRepository repo;  
    public UserService(UserRepository repo) {  
        this.repo = repo;  
    }  
  
    public User getById(Long id) {  
        return repo.findById(id)  
            .orElseThrow(() -> new NotFoundException(id));  
    }  
}
```

Test: UserServiceTest

```
@ExtendWith(MockitoExtension.class)  
class UserServiceTest {  
    @Mock  
    private UserRepository repo;  
  
    @InjectMocks  
    private UserService service;  
  
    @Test  
    void getById_WhenExists_ReturnsUser() {  
        User user = new User(1L, "John");  
        when(repo.findById(1L)).thenReturn(Optional.of(user));  
  
        User result = service.getById(1L);  
        assertNotNull(result);  
        assertEquals("John", result.getName());  
        verify(repo).findById(1L);  
    }  
}
```

Common Annotations

JUnit 5

@Test

Marks a test method

@BeforeEach

Runs before each test

@AfterEach

Runs after each test

@BeforeAll

Runs once before all tests

@AfterAll

Runs once after all tests

@DisplayName

Custom display name

Mockito

@Mock

Creates a mock

@InjectMocks

Injects mocks into the object

@Spy

Creates a spy (partial mock)

when(...).thenReturn(...)

Stubbing method behavior

verify(mock).method()

Verifies interactions

ArgumentMatchers

any(), eq(), isNull(), etc.

Unit Test Flow

1



Write Test

Create test method for the unit

2



Setup Mocks

Create mocks and define behavior

3



Inject Mocks

Inject mocks into the class under test

4



Execute

Call the method being tested

5



Verify & Assert

Verify interactions and assert outcomes

6



Test Result

View success / failure and reports



MOCKING DEPENDENCIES WITH MOCKITO · TESTING CONTROLLERS WITH MOCKMVC

· TESTING JSON REQUESTS AND RESPONSES



KEY MOCKITO FEATURES

- Create mocks easily for interfaces, classes, or concrete dependencies
- **Stub behavior:** define return values, exceptions, or custom answers
- **Verify interactions:** confirm methods were called with expected arguments and counts



BEST PRACTICES

- Mock only external dependencies, not the class under test
- Verify interactions, but don't test implementation details -- avoid over-mocking



KEY BENEFITS

- Fast and isolated -- no need to start the application context or embedded server
- **Focus on the web layer:** request mappings, validations, status codes, responses
- Rich assertions for JSON payloads, headers, and view names



BEST PRACTICES

- Use `@WebMvcTest(Controller.class)` to load only web layer components
- Mock service/repository dependencies with `@MockBean`; cover success, validation, not-found, and security scenarios



WHY IT MATTERS

- Ensures your API contract (request/response) is correct and stable
- Detects serialization, validation, and mapping issues quickly



KEY JSONPATH ASSERTIONS

- `$.field` for exact value, `$.items[*].id` for array values, `$.items.length()` for size
- Always set `contentType`, validate both happy path and error responses

KEY TAKEAWAY `@Mock` creates the fake, `@InjectMocks` wires it in, `when(...).thenReturn(...)` stubs behavior, and `verify(...)` confirms the interaction happened.

- `@WebMvcTest` loads only the web layer, `@MockBean` fakes the service, and `mockMvc.perform()` drives a real request through the controller.
- Asserting only the status code is a common pitfall; the real API contract lives in the JSON payload's fields, types, and nesting.

Spring Boot Slice Tests

Test a specific layer in isolation with focused application context



OVERVIEW



Slice tests load only a small part of the Spring application context needed for a specific layer (web, data, etc.). They are fast, focused, and ideal for unit-style tests with Spring support.

★ WHY USE SLICE TESTS?



Fast Execution

Loads minimal context → faster feedback.



Focused Testing

Test one layer at a time (web, data, JSON, etc.).



Less Configuration

Auto-configures only the components you need.



Better Isolation

Mock external dependencies easily.



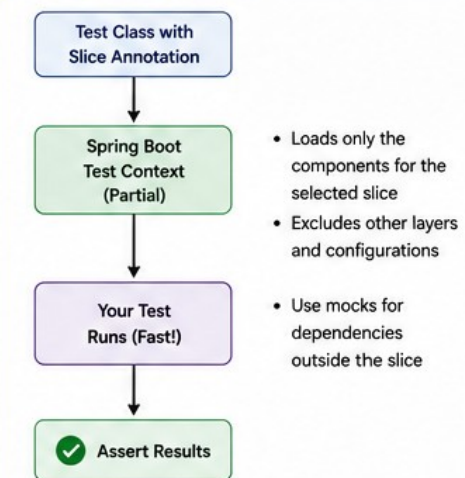
Improved Developer Productivity

Write more tests with less overhead.

COMMON SPRING BOOT SLICE ANNOTATIONS

Annotation	Focus	Loads	Use For
<code>@WebMvcTest</code>	Web Layer (Controllers)	MVC components, Controller, Jackson, Validation, MockMvc	Testing REST controllers, request mapping, JSON, validation
<code>@DataJpaTest</code>	JPA Layer (Repositories)	JPA repositories, EntityManager, H2 (embedded), JPA config	Testing Spring Data JPA repositories and queries
<code>@JsonTest</code>	JSON Layer (Serialization)	Jackson ObjectMapper, JSON components only	Testing JSON serialization/deserialization
<code>@WebFluxTest</code>	WebFlux Layer (Controllers)	WebFlux controllers, codecs, WebClient	Testing reactive controllers
<code>@JdbcTest</code>	JDBC Layer	JDBC components, DataSource, JdbcTemplate	Testing JDBC queries
<code>@DataMongoTest</code>	MongoDB Layer (Repositories)	Mongo repositories, MongoTemplate, Embedded Mongo	Testing Spring Data MongoDB repositories

HOW SLICE TESTS WORK



[J] EXAMPLE: @WebMvcTest FOR CONTROLLER TESTING

```
1 @WebMvcTest(UserController.class)
2 class UserControllerTest {
3
4     @Autowired
5     private MockMvc mockMvc;
6
7     @MockBean
8     private UserService userService;
9
10    @Test
11    void shouldReturnUser_whenUserExists() throws Exception {
12        User user = new User(1L, "John Doe", "john@example.com");
13        when(userService.findById(1L)).thenReturn(Optional.of(user));
14
15        mockMvc.perform(get("/api/users/1").accept(MediaType.APPLICATION_JSON))
16            .andExpect(status().isOk())
17            .andExpect(jsonPath("$.name").value("John Doe"))
18            .andExpect(jsonPath("$.email").value("john@example.com"));
19    }
20 }
```

OTHER SLICE EXAMPLES

```
// Test JPA Repository
@DataJpaTest
class UserRepositoryTest {
    @Autowired private EntityManager em;
    @Autowired private UserRepository repo;
    // tests...
}

// Test JSON Serialization
@JsonTest
class UserJsonTest {
    @Autowired private ObjectMapper objectMapper;
    // tests...
}
```

TIPS & BEST PRACTICES

- ✓ Use slice tests for fast and focused feedback.
- ✓ Prefer slice tests over `@SpringBootTest` for unit-style tests.
- ✓ Mock external dependencies with `@MockBean`.
- ✓ Assert both status codes and JSON content.
- ✓ Name tests clearly to describe behavior.
- ✓ Combine slices with integration tests for full coverage.
- ✓ Use `Testcontainers` with `@DataJpaTest` when needed.

WHEN TO USE WHAT?

Use Slice Tests When

- ✓ Testing a single layer
- ✓ Need fast execution
- ✓ Mocking dependencies

Use @SpringBootTest When

- ✓ Cross-layer integration
- ✓ Full context is required
- ✓ End-to-end scenarios



REPOSITORY TESTING WITH DATAJPATEST · INTEGRATION TESTING WITH SPRINGBOOTTEST



WHAT @DATAJPATEST PROVIDES

- Configures an in-memory database (H2 by default), scans @Entity and @Repository
- Configures JPA, Hibernate, DataSource; transactional tests roll back after each test
- Does NOT load the web layer, services, or other slices



WHY USE @SPRINGBOOTTEST

- End-to-end validation of the entire application stack; uses real configuration and beans
- **Tests multiple layers together:** controllers, services, repositories, infrastructure
- Works with embedded databases or Testcontainers for real services



BEST PRACTICES

- Use TestEntityManager for fine-grained JPA control; clear the persistence context to test real state
- Combine @DataJpaTest with Testcontainers for higher-fidelity database engines

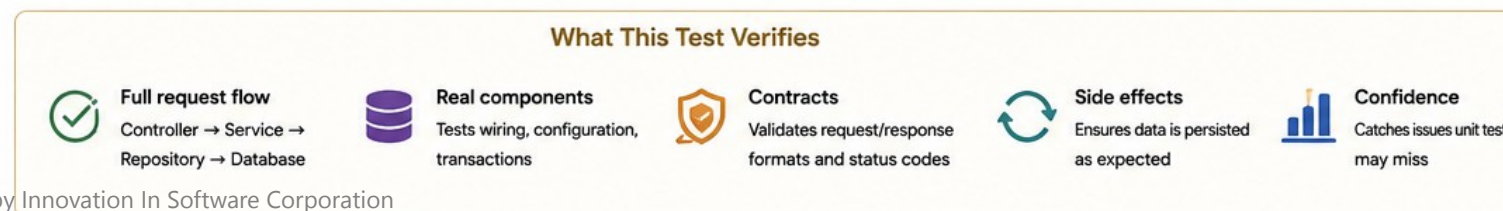
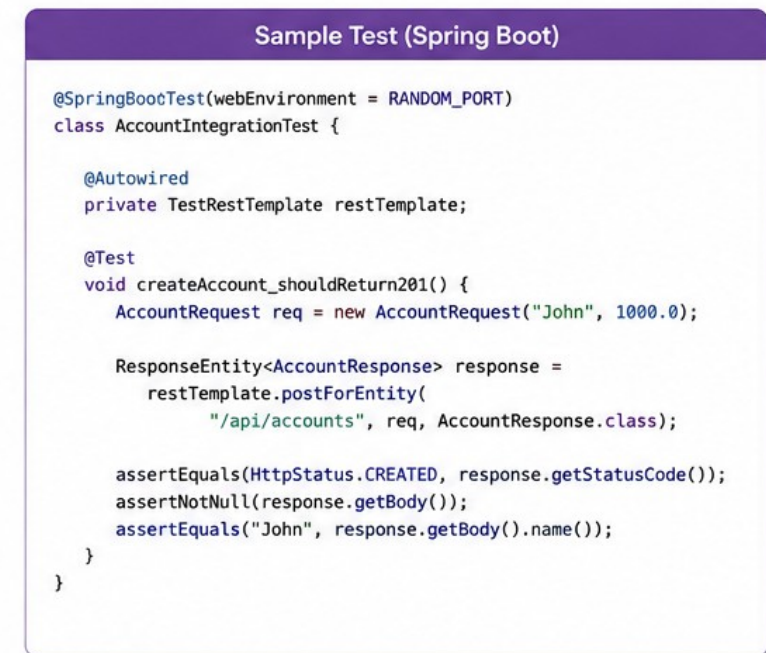
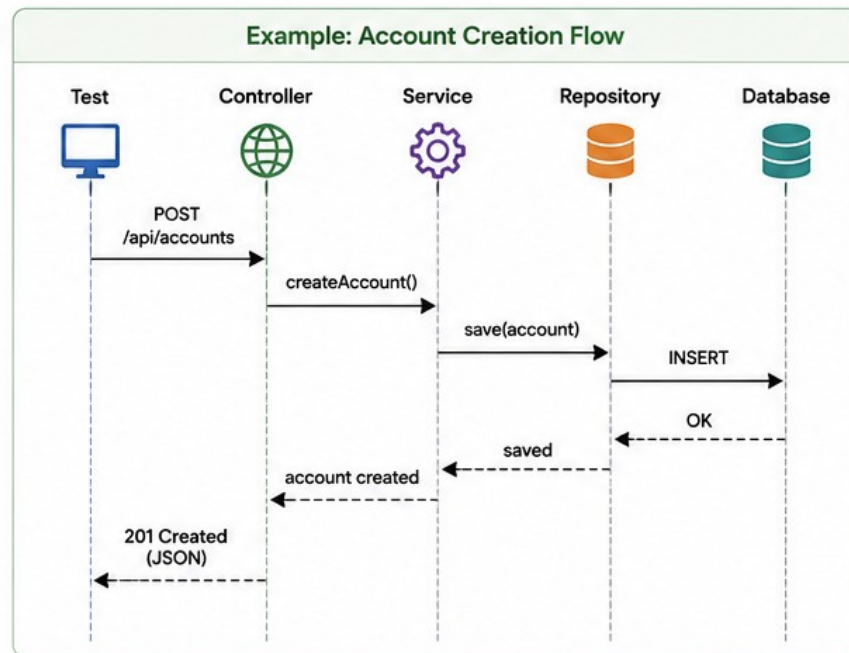
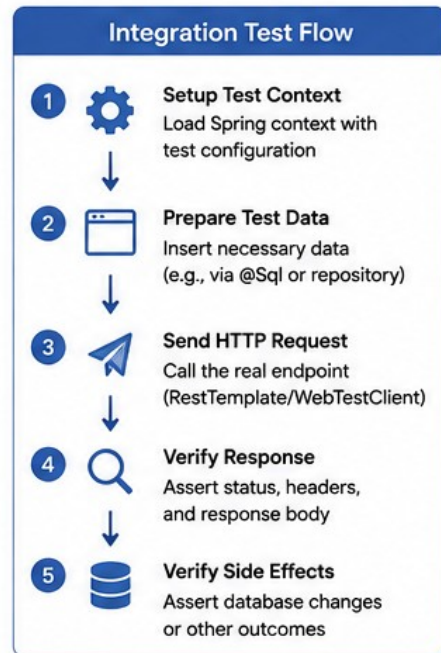
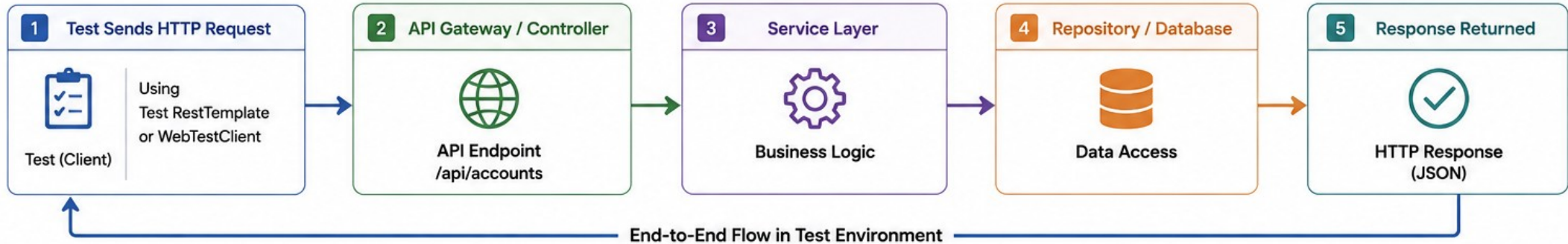


BEST PRACTICES

- Use @SpringBootTest sparingly; prefer slice tests for unit scope
- Use @ActiveProfiles("test") for test-specific config; isolate and clean up test data

KEY TAKEAWAY TestEntityManager gives fine-grained control to persist, flush, and clear the persistence context so queries hit the real database, not a cache. • Controllers, services, repositories, and infrastructure all run together -- catching integration issues a slice test can never see.

One Guided Integration Test



TESTING EXTERNAL SERVICE CALLS · TESTCONTAINERS OVERVIEW



TECHNIQUES TO TEST EXTERNAL CALLS

- **Mock with @MockBean:** replace the HTTP client bean and stub responses
- **MockServer / WireMock:** a lightweight HTTP server returning stubbed JSON responses
- **Testcontainers + real service:** run the real external service in a container for higher fidelity



KEY BENEFITS

- Real dependencies -- test against the real thing, not mocks
- **Isolated and reproducible:** clean containers for every test run
- Works locally and in CI, with automatic lifecycle management



BEST PRACTICES

- **Test both happy path and failure scenarios:** timeouts, 4xx/5xx errors, malformed responses
- Keep tests deterministic and independent

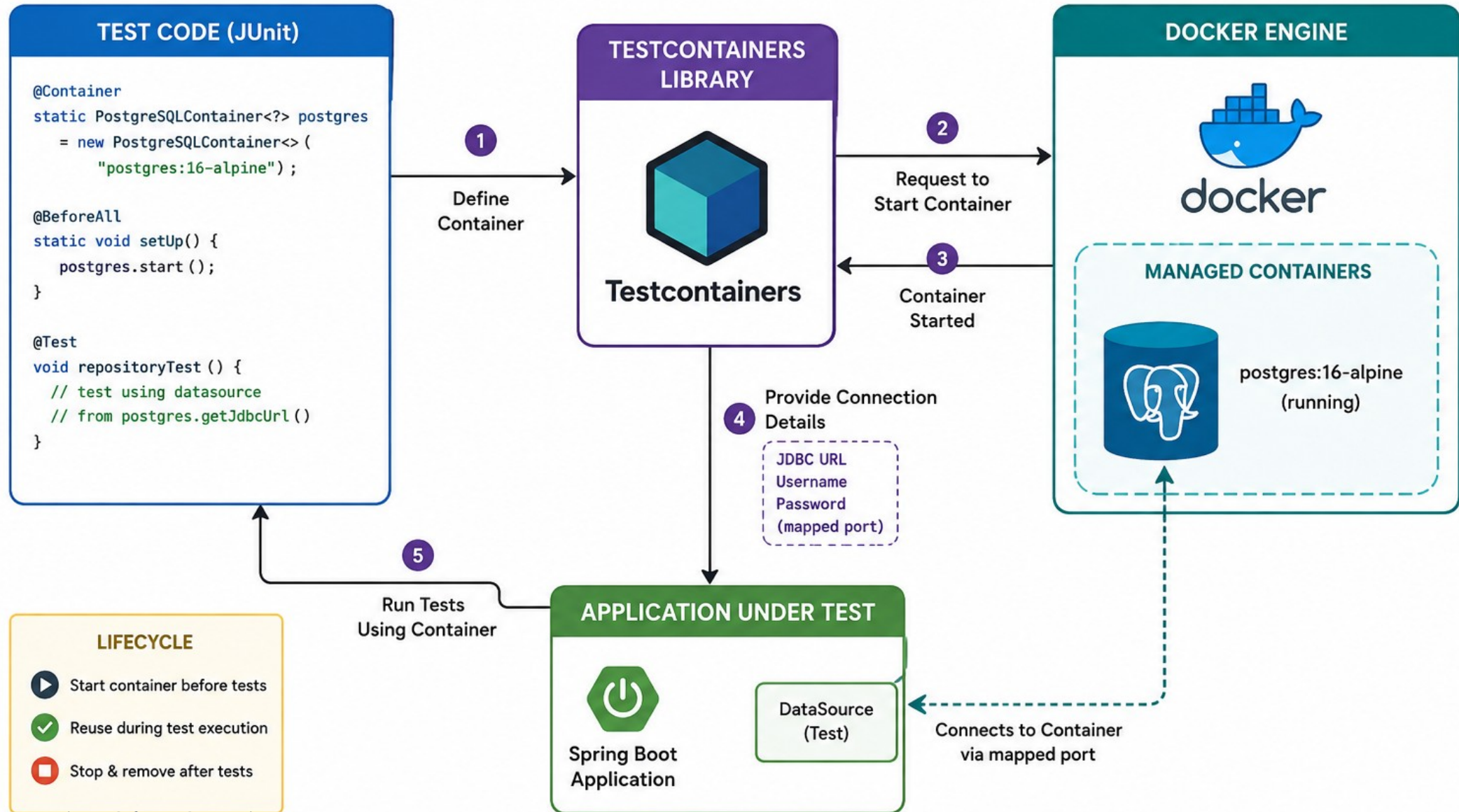


COMMONLY USED MODULES

- PostgreSQL and Kafka are the classroom stores; Redis/RabbitMQ modules exist in Spring but are lecture-only here

KEY TAKEAWAY Mocking with @MockBean, running MockServer/WireMock, or spinning up the real service in a container each avoid flaky, costly, real external calls. • Throwaway, isolated, and reproducible environments replace mocks with the real infrastructure your service actually depends on.

TESTCONTAINERS AWARENESS (IF ENVIRONMENT PERMITS)



Database Integration Tests with Testcontainers

Test your persistence layer against a real database in an isolated container



WHY USE TESTCONTAINERS FOR DATABASE TESTS?

- ✓ Uses a real database – not an in-memory substitute
- ✓ Reproducible and isolated – clean DB for every test run
- ✓ Works locally and in CI – same behavior everywhere
- ✓ Matches production – same engine, version, and config
- ✓ Catches integration issues early – SQL, mappings, constraints

HOW IT WORKS (FLOW)



EXAMPLE (SPRING BOOT + JPA + TESTCONTAINERS)

```
@Testcontainers
@DataJpaTest
class UserRepositoryIT {

    @Container
    static PostgreSQLContainer<?> postgres =
        new PostgreSQLContainer<>("postgres:16-alpine")
            .withDatabaseName("testdb")
            .withUsername("test")
            .withPassword("test");

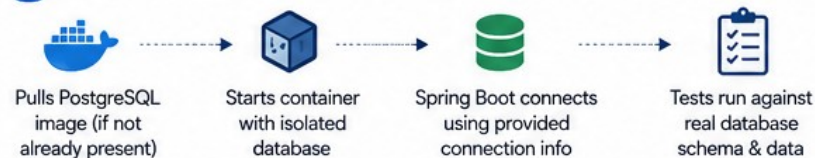
    @DynamicPropertySource
    static void configureProperties(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
        registry.add("spring.datasource.driver-class-name", postgres::getDriverClassName);
    }

    @Autowired
    private UserRepository userRepository;

    @Test
    void shouldSaveAndFindUser() {
        User user = new User(null, "Alice", "alice@example.com");
        userRepository.save(user);

        Optional<User> found = userRepository.findByEmail("alice@example.com");
        assertTrue(found.isPresent());
        assertEquals("Alice", found.get().getName());
    }
}
```

WHAT HAPPENS?



CONFIGURATION OPTIONS

- Use different DB versions/images
- Set init scripts:

```
.withInitScript("schema.sql")
.withInitScript("data.sql")
```

- Reuse containers for faster builds:

```
testcontainers.reuse.enable=true
```

BENEFITS

- ✓ Real database behavior (constraints, indexes, transactions, isolation)
- ✓ Validates SQL and JPA mappings
- ✓ Detects dialect-specific issues
- ✓ Eliminates "it works on my machine" problems

BEST PRACTICES

- Use @DataJpaTest for repository slice tests
- Use @SpringBootTest for full-stack DB tests
- Keep containers lightweight (alpine images)
- Use init scripts for schema/migrations if needed
- Avoid hardcoded ports – let Testcontainers manage
- Reuse containers locally to speed up test runs



KEY TAKEAWAY

Testcontainers brings real infrastructure to your tests. For database integration tests, it provides confidence that your data layer works exactly as expected in production.



RESULT: High-quality, reliable tests that catch integration issues early and reduce production defects.

MESSAGING INTEGRATION TESTS · CONTRACT TESTING FUNDAMENTALS



WHY USE TESTCONTAINERS FOR MESSAGING

- Tests can run against a real Kafka broker when Testcontainers is available -- otherwise use the classroom Compose broker
- Validates message serialization/deserialization and consumer acknowledgments
- Catches configuration and connectivity issues early



BEST PRACTICES

- Inject connection info via `@DynamicPropertySource`; use unique topics/queues per test
- Use Awaitility for async verifications; keep tests isolated and idempotent



HOW CONTRACT TESTING WORKS

1. Consumer creates a contract defining expected requests and responses
2. The contract (Pact file) is published to a contract repository
3. Provider runs contract tests to verify it meets all consumer expectations
4. If verification passes, the provider can be deployed with confidence

KEY TAKEAWAY Awareness: Testcontainers can run Kafka. Lab 3's guided integration test uses the classroom database unless the VM already supports Testcontainers. • The consumer defines expected requests and responses; the provider verifies it can satisfy that same contract -- both sides trust one shared Pact file.

CONSUMER-DRIVEN CONTRACT TESTING

A Pact Broker becomes the shared source of truth, letting the consumer and provider teams release independently without breaking each other.



WHY IT MATTERS

- Reduces integration failures by catching breaking changes before deployment
- Enables independent development -- consumer and provider work in parallel
- Provides living documentation that's always up to date with real usage



BEST PRACTICES

- Start with consumers -- write contract tests first
- Use a contract broker for sharing and discovery; verify contracts in CI for every provider change

KEY TAKEAWAY A Pact Broker becomes the shared source of truth, letting the consumer and provider teams release independently without breaking each other.

API COMPATIBILITY TESTING · END-TO-END TESTING · TEST DATA MANAGEMENT

TYPES OF COMPATIBILITY

- **Backward compatible:** new API works with existing consumers without changes
- **Forward compatible:** existing API can work with newer consumers
- **Breaking change:** existing consumers may fail with the new API

BEST PRACTICES

- Start with consumer-driven contracts; run provider verification in CI on every change
- Use semantic versioning for APIs; automate compatibility checks as a quality gate

WHAT E2E TESTING VALIDATES

- Complete workflows and integrations across service boundaries
- Data integrity, configuration, security, and environment issues
- Confidence before production releases

BEST PRACTICES

- Focus on critical business flows, not every path; keep tests independent and idempotent
- Tag tests (e.g., @E2E) and run less frequently than unit tests

TEST DATA LIFE CYCLE

- Identify -> Create/Acquire -> Store/Manage -> Provision -> Use & Refresh -> Retire/Archive

BEST PRACTICES

- Use Testcontainers to spin up clean, isolated databases
- Mask/anonymize sensitive data (PII) before use; clean up data after tests to avoid contamination

KEY TAKEAWAY Comparing a new API version against its baseline catches whether an added field, removed field, or changed type will break existing consumers. • Order creation flowing through the API gateway, order, inventory, payment, and notification services proves the whole system works together. • Production data must be masked before use; synthetic and reference data avoid that risk entirely while still exercising real scenarios.

CONTINUOUS TESTING IN CI-CD · CODE COVERAGE AND TEST QUALITY · AVOIDING BRITTLE TESTS



TESTING THROUGHOUT THE PIPELINE

- Code Commit -> Build -> Unit Test -> Integration Test -> Contract Test -> Packaging -> Deploy -> E2E/Smoke Test
- Each stage has its own quality gate and verification focus



KEY PRINCIPLES

- **Shift left:** test as early as possible, when defects are cheapest to fix
- Fast feedback, automated end-to-end coverage, and quality gates at every stage



GOOD COVERAGE TARGETS (GUIDELINES)

- **Unit tests:** 80-90% on core business logic and critical paths
- **Integration tests:** 70-80%; overall project: 70%+; critical modules: 90%+



COVERAGE VS. QUALITY

- **High coverage, low quality:** fragile, over-mocked tests give a false sense of security
- **Good quality, meaningful coverage:** resilient, focused tests that validate real behavior



WHY TESTS BECOME BRITTLE

- **Overly coupled to implementation:** depending on private methods, internal state, or call order
- Excessive mocking creates false confidence and breaks on refactoring
- Time and environment dependencies make tests unstable



STRATEGIES FOR RESILIENT TESTS

- Test behavior, not implementation; favor real components over mocks
- Control external factors with fixed clocks, Testcontainers, and in-memory DBs

KEY TAKEAWAY Unit, integration, contract, and end-to-end tests each gate a different stage -- from code commit through packaging to deployment. • 80-90% unit, 70-80% integration, and 90%+ on critical modules are sensible coverage targets -- but meaningful coverage matters more than 100%. • Verifying internal method calls, over-mocking, and time-dependent assertions are the three most common ways tests become brittle.

PRODUCTION-READINESS TESTING CHECKLIST

Functional, data, integration, resilience, performance, security, observability, configuration, quality, and deployment readiness -- all ten, signed off.



CHECKLIST CATEGORIES (1-5)

- 1. Functional testing:** business rules and API contracts behave as specified for valid and invalid input
- 2. Data & persistence testing:** migrations apply cleanly and repositories read and write what you expect
- 3. Integration testing:** the service works against real collaborators, not only against mocks
- 4. Resilience & reliability testing:** timeouts, retries, breakers, and fallbacks behave under failure
- 5. Performance testing:** latency and throughput hold at expected load, with headroom for peaks



CHECKLIST CATEGORIES (6-10)

- 6. Security testing:** authentication, authorisation, and input validation are enforced, not assumed
- 7. Observability testing:** logs, metrics, and traces actually appear and carry correlation IDs
- 8. Configuration & environment:** one artifact runs everywhere, configured only from outside
- 9. Quality & test coverage:** coverage is meaningful on the paths that carry risk, not just a percentage
- 10. Deployment readiness:** probes, resource limits, rollout strategy, and rollback are all in place

KEY TAKEAWAY Functional, data, integration, resilience, performance, security, observability, configuration, quality, and deployment readiness -- all ten, signed off.

EXERCISE 3.3 — ADD SECURITY AND RESILIENCE TESTS

Module 8 checkpoint C — Unit + one integration test.



OBJECTIVE

- Unit + one integration test
- 10 min · code warmup checkpoint
- **Worksheet:**
curriculum/day-03/exercises/exercise-3.3-security-resilience-tests.md



DO THIS

1. Unit-test 401/403 on a protected Account or Transaction endpoint
2. Unit-test circuit-open → 503 fallback (no fake ACTIVE account)
3. Name the one integration test you will run (persistence or API with the classroom DB)
4. Testcontainers is awareness only unless the VM already supports it



EXPECTED RESULT

- A short test list that Lab 3 can tick. Coverage of security and the fallback rule.
- Lab 3 adds unit tests plus one guided integration test (AccountPersistenceTest).

KEY TAKEAWAY A short test list that Lab 3 can tick. Coverage of security and the fallback rule.

DAY 3 SUMMARY

Secure, Resilient & Production-Ready Services

WHAT WE LEARNED



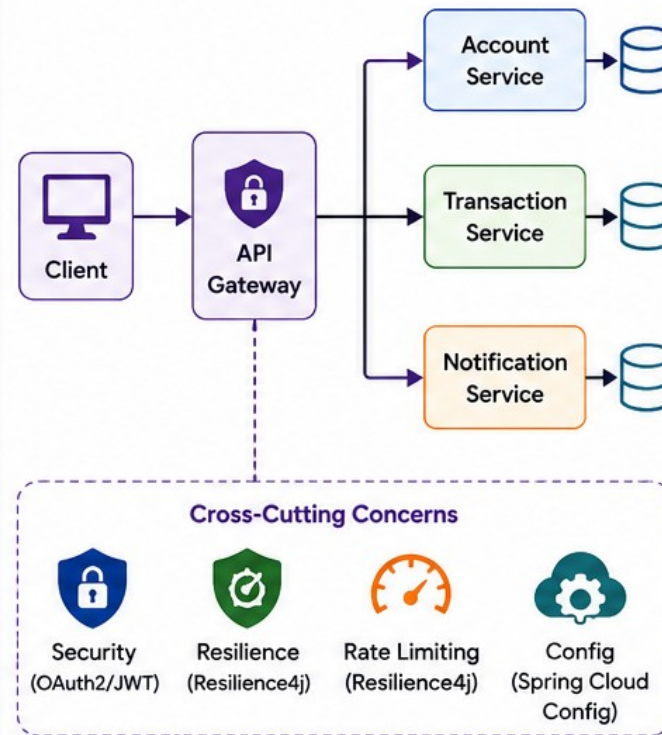
- Securing services using OAuth2 Resource Server & JWT
- Building resilience with Resilience4j (Circuit Breaker, Retry, Timeout)
- Protecting services with Rate Limiting
- Externalizing configuration with Spring Cloud Config
- Observability with Actuator, Prometheus & Grafana
- Testing strategies with Testcontainers

KEY TECH & TOOLS



- Spring Security**
OAuth2 / JWT
- Resilience4j**
Circuit Breaker, Retry, Timeout, Rate Limiter
- Spring Cloud Config**
Centralized Configuration
- Spring Boot Actuator**
Health, Metrics, Endpoints
- Prometheus**
Metrics Collection
- Grafana**
Dashboards & Visualization
- Testcontainers**
Reliable Integration Testing

WHAT WE BUILT



OUTCOMES



- ✓ Secured APIs with OAuth2 and JWT validation
- ✓ Resilient services that handle failures gracefully
- ✓ Protected APIs from abuse with rate limiting
- ✓ Centralized and dynamic configuration management
- ✓ Observable services with metrics and dashboards
- ✓ Reliable integration tests using Testcontainers

KEY TAKEAWAYS



Security is essential



Resilience improves reliability



Rate limiting protects services



Centralized config simplifies operations



Observability drives insights



Testing ensures confidence

WHAT'S NEXT



Continue to Lab 4:
Event-Driven Architecture
& Asynchronous Communication



KNOWLEDGE CHECK

1 What is the primary purpose of OAuth2 Resource Server?



- ☐ A To authenticate users locally
- ☐ B To validate JWTs and protect APIs
- ☐ C To issue access tokens
- ☐ D To encrypt request payloads

2 Which Resilience4j component prevents cascading failures?



- ☐ A Retry
- ☐ B Circuit Breaker
- ☐ C Rate Limiter
- ☐ D Timeout

3 What is the role of a Rate Limiter?



- ☐ A Encrypt HTTP traffic
- ☐ B Limit the number of requests over a period of time
- ☐ C Validate user roles
- ☐ D Cache responses

4 Which Spring Cloud Config feature allows centralized management of application properties?



- ☐ A Config Client
- ☐ B Config Server
- ☐ C Config Actuator
- ☐ D Config Repository

5 Which Actuator endpoint exposes application metrics by default?



- ☐ A /health
- ☐ B /info
- ☐ C /metrics
- ☐ D /loggers

6 Which tool is used to visualize metrics collected from services?



- ☐ A Kibana
- ☐ B Prometheus
- ☐ C Zipkin
- ☐ D Keycloak

7 What is Testcontainers primarily used for?



- ☐ A Container orchestration in production
- ☐ B Spinning up real dependencies in tests
- ☐ C Building Docker images
- ☐ D Monitoring container logs

8 Which HTTP status code indicates that the request was successful?



- ☐ A 400
- ☐ B 401
- ☐ C 200
- ☐ D 500

KEY CONCEPTS COVERED



Security
(OAuth2, JWT)



Resilience
(Resilience4j)



Rate Limiting
& Protection



Centralized
Configuration



Observability
(Actuator, Metrics)



Monitoring
(Prometheus, Grafana)



Testing
(Testcontainers)

LAB 3

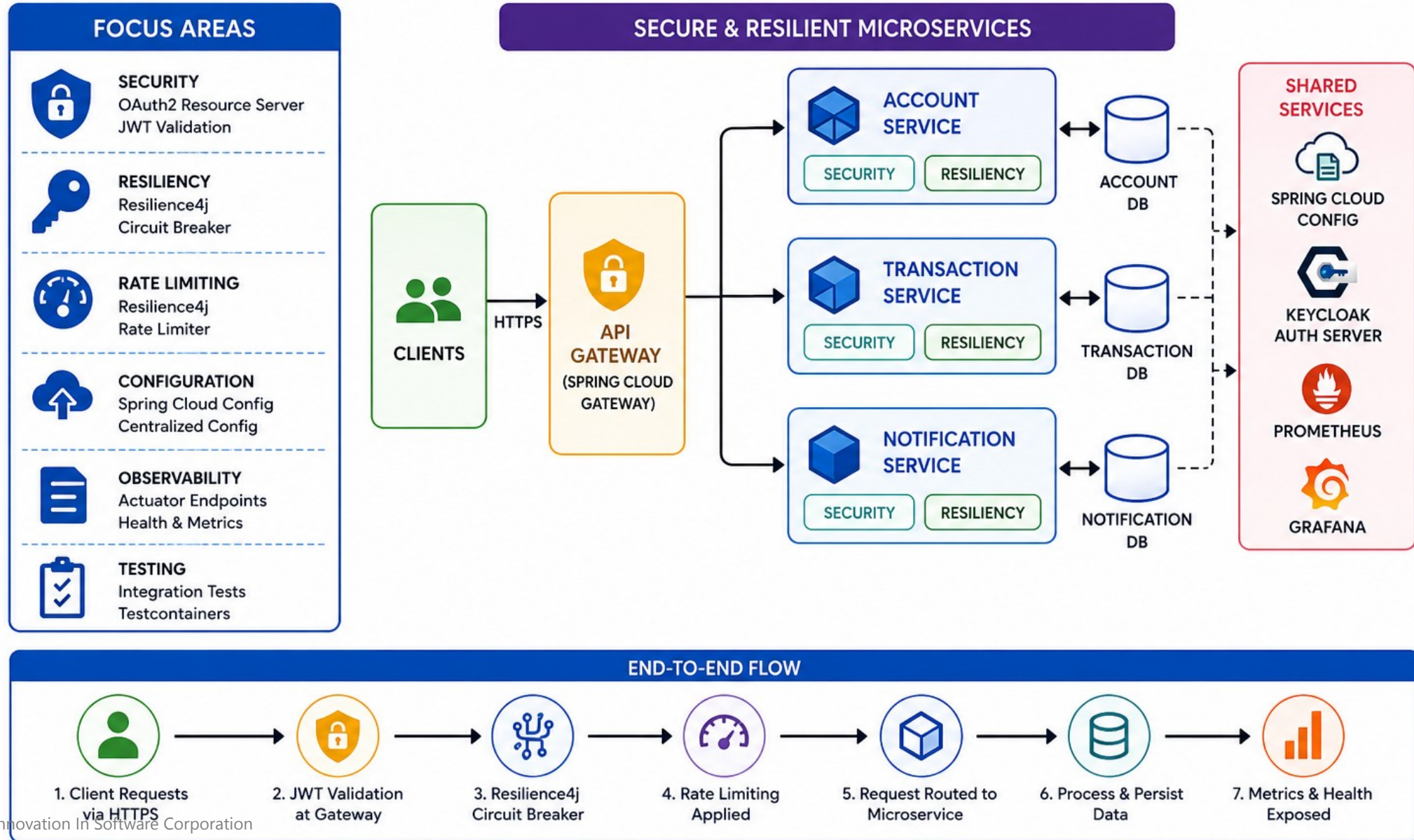
Lab 3 — Secure and Resilient Services

Secured, resilient, tested Account and Transaction APIs with safe fallbacks.

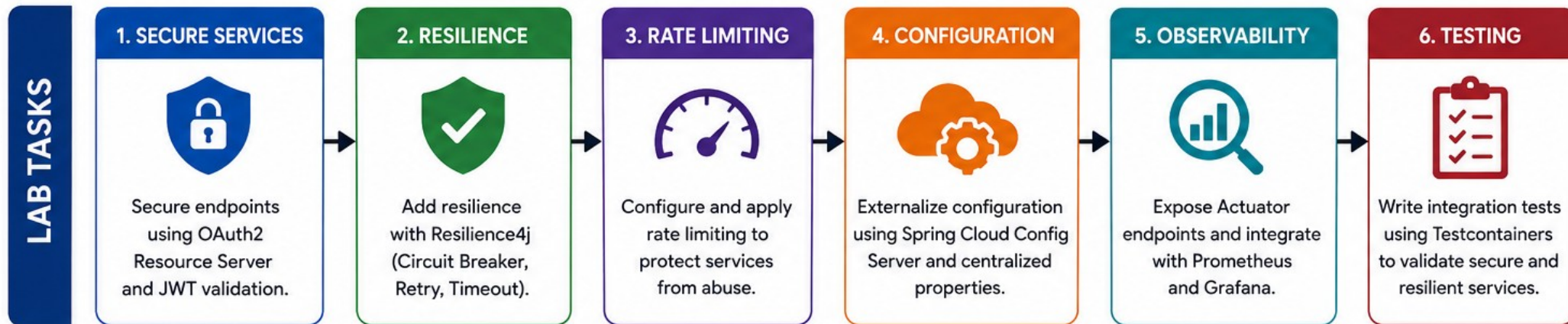


LAB 3 OVERVIEW

SECURE AND RESILIENT SERVICES



LAB TASKS AND DELIVERABLES



DELIVERABLES	SECURITY DELIVERABLES	RESILIENCE DELIVERABLES	RATE LIMITING DELIVERABLES	CONFIGURATION DELIVERABLES	OBSERVABILITY DELIVERABLES	TESTING DELIVERABLES
	 • OAuth2 Resource Server configured • JWT validation implemented • Secured endpoints verified	 • Resilience4j integrated in services • Circuit Breaker configured • Fallback behavior implemented and tested	 • Rate Limiter configured with policies • Applied to selected endpoints • Rate limiting behavior verified	 • Spring Cloud Config Server set up • Services reading config from Config Server • Profiles and properties externalized	 • Actuator endpoints exposed • Prometheus scraping configured • Grafana dashboards created	 • Integration tests written with Testcontainers • Key scenarios covered (security, resilience, rate limiting) • All tests passing



LAB 3 — SECURE AND RESILIENT SERVICES — OVERVIEW

Day 3 lab: what you build and how it grows the capstone.



WHAT YOU WILL FINISH WITH

- Both APIs reject callers without a Bearer token (401)
- Wrong scope returns 403 (not a silent 200)
- Transaction Service forwards the caller's JWT when it GETs an account
- If Account Service is down, POST /api/v1/transactions returns 503 ACCOUNT_SERVICE_UNAVAILABLE — no Kafka event, no fake ACTIVE account
- After enough failures the circuit opens; later calls fail fast
- Unit tests for security and the circuit breaker
- Logs still use synthetic ids only — no Authorization header, no token payload

KEY TAKEAWAY Lab 3 builds Secure and Resilient Services -- the working code behind today's concepts, and the next layer of the banking capstone.

LAB 3 — SECURE AND RESILIENT SERVICES — BEFORE YOU START

Check these before the lab clock starts.

 KNOWLEDGE YOU NEED (FROM DAY 3)

Day 3 idea	How it appears in this lab
Resource server	Each API validates a JWT. It does not log users in with a form.
Scopes	accounts.read / accounts.write and transactions.read / transactions.write.
Workload vs user	The human caller holds the JWT. Transaction Service relays that token to Account Service.
Timeout	RestClient already uses a 2s connect / 3s read timeout. That is the timeout.
Circuit breaker	Resilience4j wraps AccountClient.requireActiveAccount. Repeated failures open the circuit.
Safe fallback	Fallback throws 503. It must not return a synthetic ACTIVE account.
Log redaction	Log accountId and status. Never log Bearer tokens or JWT claims.

KEY TAKEAWAY Settle prerequisites and environment first -- most lab time is lost to setup, not to code.

LAB 3 — SECURE AND RESILIENT SERVICES — LAB STEPS

6 steps, worked in order.

LAB STEPS

- Step 1 — Start both services and prove they are still open
- Step 2 — Protect Account Service with JWT
- Step 3 — Protect Transaction Service the same way

LAB STEPS (CONTINUED)

- Step 4 — Relay the Bearer token to Account Service
- Step 5 — Circuit breaker and safe fallback
- Step 6 — Tests and log review

KEY TAKEAWAY Work the steps in order -- each one builds on the last.

LAB 3 — SECURE AND RESILIENT SERVICES — SUCCESS CRITERIA

You are done when every box below is ticked.

SUCCESS CRITERIA

- No token → 401 on both business APIs
- Teller cannot POST transactions (403)
- Ops can POST a transaction for an ACTIVE account
- Token is forwarded; Account GET is authorized

SUCCESS CRITERIA (CONTINUED)

- Account down → 503, no auto-approve
- Circuit opens after repeated failures
- mvn test passes on both services (includes the guided integration test AccountPersistenceTest)
- Logs stay synthetic and secret-free

KEY TAKEAWAY Tick every criterion before moving on -- the next lab builds directly on this one.